

SDS User Program Documentation

Anthony D. Appelhans and David A. Dahl*

Introduction

SDS (Statistical Diffusion Simulation) is a user program for SIMION that simulates motions of charged particles in atmospheric pressure conditions in **electrostatic** and **magnetic** (see the special magnetic issues at the end of the appendix) fields. It avoids the computationally intensive issues of high collision rates by employing collision statistics to simulate the effects of millions of collisions per time step. Ion motions are simulated by a combined viscous ion mobility and random ion jumping approach.

This document provides an overview of SDS with details in the appendix. The distribution copy of SDS comes in a directory called SDS plus four attached demo subdirectories. It is suggested that you scan read the eight main pages in this document first. Next, read the demo documents and run the demos, starting with the demo in the SDS directory. Although SDS may appear to be somewhat complex, it is quite trivial to use at an entry level (e.g. SDS subdirectory demo). Have fun!

Ion Mobility

The average motion (or drift) of each ion is simulated via ion mobility (using an accel_adjust user program segment (that can also support local bulk gas velocity fields)). This is a viscous effect where the force of the electric field is balanced by drag created by the ion's many collisions with the bath gas:

$$\text{Ion mobility velocity:} \quad v_m (\text{cm/s}) = K \frac{d\text{Volts}}{d\text{length}} = K * E$$

$$\text{Where:} \quad K = \text{mobility} (10^{-4} \text{ m}^2 \text{ Volt}^{-1} \text{ sec}^{-1})$$
$$E = \text{electric field strength (Volts/m)}$$

The mobility coefficient K is a function of local temperature and pressure. To simplify this effect, a reduced mobility K_0 is employed that represents the ion's mobility at STP (standard temperature and pressure: $T = 273.15$ °K and $P = 760$ Torr). The following formula can be employed to convert K_0 to the K at the operating pressures and temperatures:

$$\text{Measured K from reduced } K_0: \quad K = K_0 \left(\frac{760}{P} \right) \left(\frac{T}{273.15} \right)$$

$$\text{Where} \quad K = \text{measured K}$$
$$K_0 = \text{reduced K}$$

(*) This documentation and the user program described are based on the original SDS model by Dave Dahl, 2004-09-27, included in PRG format in the supplementary material in the paper: *Anthony D. Appelhans and David A. Dahl. SIMION ion optics simulations at atmospheric pressure International Journal of Mass Spectrometry. Volume 244, Issue 1, 15 June 2005, Pages 1-14. [<http://dx.doi.org/10.1016/j.ijms.2005.03.010>].* The code and documentation were adapted by SIS and are included in SIMION 8 by permission of Appelhans and Dahl.

SDS User Program Documentation

T = measured temperature (°K)

P = measured pressure in Torr (mm Hg)

The SDS user program assumes that ANY ion mobility supplied by the user is a reduced mobility K_0 ($10^{-4} \text{ m}^2 \text{ V}^{-1} \text{ s}^{-1}$) **appropriate for the collision gas specified**. The units required by SDS for K_0 are the standard units used within the ion mobility community.

Statistically Simulated Diffusion

SDS uses collision statistics to simulate hard sphere collision based diffusion of the ions (using an other_actions user program segment). Statistically randomized jumps are superimposed on the ion's drift location to combine the drift and diffusion effects.

An brief overview of this process is as follows: SDS uses the ion's mass (amu - specified) and diameter (nm - specified or estimated) along with the bath gas's mass (amu - specified) and diameter (nm - specified) as well as the local temperature and pressure to estimate the ion's average thermal speed and mean free path (MFP) using basic hard sphere kinetic gas theory. SDS then uses these factors and the value of the current integration time step to predict the number of expected collisions in the current time step.

A random number is used to obtain a standardized ion jump radius via interpolation between two appropriate statistic tables (selected by the $\text{mass}_{\text{ion}}/\text{mass}_{\text{gas}}$ ratio from five tables for mass ratios from 1 to 10,000). This standardized jump radius is then scaled by the ion's MFP and the square root of the ratio of the expected number of collisions to the number of collisions represented in each collision statistic table. The result is the radius (mm) to jump the ion. The other_actions user program segment then jumps the ion the specified radius in a random direction from its current drift location. A more detailed explanation of the statistical method employed can be found in the appendix at the end of this document.

Parameter Specification

The SDS user program provides a collection of adjustable variables that the user can initialize before each Fly'm. Below is a list of these variables and their default values:

Variable Name	Default	Meaning
_Use_Collision_Gas_Mass_amu	28.94515	normal air mixture
_Use_Collision_Gas_Diameter_nm	0.366	normal air mixture
_Use_Pressure_torr	760	sea level (101.325 Pa)
_Use_Temperature_K	298.15	25 °C
_Use_Vx_m_per_sec	0.0	no bulk gas velocity in x
_Use_Vy_m_per_sec	0.0	no bulk gas velocity in y
_Use_Vz_m_per_sec	0.0	no bulk gas velocity in z
_Use_P_field_scale_factor	1.0	use any P field as defined
_Use_T_field_scale_factor	1.0	use any T field as defined

<code>_Use_V_field_scale_factor</code>	1.0	use any V field as defined
<code>_Use_min_time_step_usec</code>	0.0	no minimum to time step

The default values should be appropriate for your initial runs with SDS.

The remaining unknowns for each ion are K_0 ($10^{-4} \text{ m}^2 \text{V}^{-1} \text{s}^{-1}$) and its collision diameter (nm). For the moment, these can be ignored, because SDS will automatically estimate them for you from each ion's mass.

A Quick Start

SDS also allows the user to define ion parameters K_0 and d keyed to ion mass as well as provide spatially varying fields of pressure, temperature, and bulk gas velocity. However, no knowledge of these matters is required to make simple but useful runs with the SDS user program.

Let's assume that you have an existing simulation (**WITHOUT USER PROGRAMS**) that uses a fast adjust potential array. Assume the PA definition file is TEST.PA# and the workbench file is SYSTEM.IOB. The following two files **MUST** be copied into your simulation's subdirectory to be able to run SDS with your simulation (e.g. from the SDS subdirectory):

<code>collision_sds.lua</code>	This is the working copy of the SDS user program
<code>MBMR.DAT</code>	Collision statistic for a mass ratio of 1, 10, 100, 1000, and 10000

These files are optional:

<code>M_DEFS.DAT</code>	K_0 and d defined by mass (two defined by default)
<code>P_DEFS.DAT</code>	Spatial pressure field (empty by default)
<code>T_DEFS.DAT</code>	Spatial temperature field (empty by default)
<code>Vx_DEFS.DAT</code>	Spatial bulk gas x velocity field (empty by default)
<code>Vy_DEFS.DAT</code>	Spatial bulk gas y velocity field (empty by default)
<code>Vz_DEFS.DAT</code>	Spatial bulk gas z velocity field (empty by default)

Now that you have copied the above eight files, either rename `collision_sds.lua` as `SYSTEM.lua` or copy it to a new file called `SYSTEM.lua` in the simulation's subdirectory. **Important: If you are using a *magnetic* potential array the SDS user program **MUST** only be active in the magnetic PA instead of the electrostatic array. This is because magnetic accelerations are computed last and SDS needs to be applied to the total acceleration (this can be accomplished by ensuring the magnetic PA is instance #1 in the PA instance list in the PAs tab).** Next, run SIMION, load the IOB file for your simulation, set the trajectory quality to zero (or a small negative number), and Fly'm. An interesting way to view the ions is to fly them grouped as dots. Use groups of ions with different masses. Limit the total number of ions to fewer than 200 ions for speedy runs.

The SDS Data Header

Whenever SDS executes from within SIMION it outputs a data header block just before it flies the ions. This data header contains a lot of interesting and potentially useful information. The illustration below shows the SDS data header that is created when the demo in the SDS directory executes. The first thing that SDS does is to look for any nonempty pressure, temperature, and or bulk velocity arrays (discussed below). In the case of this demo no nonempty P, T, or V arrays are active.

Next, SDS loads the Mass definition data from the M_DEFS.DAT file. In this case it found two mass definitions in the file. One is for ions of mass 190 amu and the other is for ions of mass 130 amu. The M_DEFS.DAT file contains user values for the diameter (nm) and STP (standard temperature – 273.15 °K and pressure – 760 torr) reduced mobility K_0 ($10^{-4} \text{ m}^2 \text{ v}^{-1} \text{ s}^{-1}$) for these ions. Because they were defined in the file the term (def) appears behind each number. If you only supplied the diameter then SDS would estimate the K_0 and place the term (est) after the K_0 number (more of this below). SDS also computes two more numbers: MFPo is the STP mean free path (mm), and Vo is the STP average thermal ion velocity (mm/ μsec).

```

Initializing user program arrays and parameters
Checking P, T, and Vel arrays for data
Note: No P, T, or V arrays active

Mass Definitions loaded or created
N, mass, dia, Ko Mobility, MFPo, Avg Vo
(n), (amu), (nm), (10-4 m2V-1s-1), (mm), (mm/ $\mu$ sec)
n= 1, m= 190, d= 0.69(def), Ko= 1.36(def), MFPo= 1.882e-005, Vo= 0.1745
n= 2, m= 130, d= 0.61(def), Ko= 1.56(def), MFPo= 2.497e-005, Vo= 0.2109

Added: No match found in MASSDEFS.DAT
n= 3, m= 400, d= 0.8872(est), Ko= 0.9733(est), MFPo= 9.967e-006, Vo= 0.1202
n= 4, m= 28, d= 0.3657(est), Ko= 3.076(est), MFPo= 6.279e-005, Vo= 0.4545
n= 5, m= 1.5e+004, d= 2.97(est), Ko= 0.1494(est), MFPo= 2.217e-007, Vo= 0.01964
----- Begin Next Fly'm -----

Local Values for ions at user defined:
Pressure = 760 Torr
Temperature = 298.1 K

mass, K Mobility, MFP, Avg V, B1 MFP/rB1=1
(amu), (10-4 m2V-1s-1), (mm), (mm/ $\mu$ sec), (gauss)
m= 28, K= 3.357, MFP= 6.854e-005, v= 0.4748, B1= 2.01e+007
m= 130, K= 1.703, MFP= 2.726e-005, v= 0.2204, B1= 1.089e+008
m= 190, K= 1.484, MFP= 2.054e-005, v= 0.1823, B1= 1.747e+008
m= 400, K= 1.062, MFP= 1.088e-005, v= 0.1256, B1= 4.787e+008
m= 1.5e+004, K= 0.1631, MFP= 2.42e-007, v= 0.02051, B1= 1.318e+011

```

If ions are defined that do not match masses defined in the M_DEFS.DAT file, SDS will estimate values for them. Notice in the example above, SDS automatically created three ion mass definitions for ions of mass 400, 28, and 15,000 amu. Note that (est) is associated with their d and K_0 estimates.

SDS also looks at the volume wide pressure and temperature that the user specified (or defaulted to – see just below). It then computes a sorted table (by mass) of the ions it actually found and their local working temperature and pressure values for K (mobility), MFP (mean free path), V (average thermal speed), and B1.

B1 represents a significant magnetic field transition. It is the magnetic field (in gauss) required to have the ion's thermal cyclotron radius equal its mean free path. Magnetic field strengths above B1 can cause SDS to overestimate molecular diffusion, because ion motions start to become a collection of significant circular arcs and/or circles. More of this is discussed at the end of the appendix below.

Interesting Things to Try

It is assumed that you have now flown ions using an SDS user program (e.g. the demo in the SDS subdirectory). When pressure, temperature and bulk gas velocity are NOT defined as spatially varying fields (the default situation), their respective adjustable variables can be used to set volume wide values.

For example you can change the volume wide pressure and temperature by adjusting the values of the pressure and temperature adjustable variables below:

Variable Name	Default	Meaning
<code>_Use_Pressure_torr</code>	760	sea level (101.325 Pa)
<code>_Use_Temperature_K</code>	298.15	25 °C

You can also set a uniform bulk gas flow throughout the volume changing the values of the following three adjustable variables:

Variable Name	Default	Meaning
<code>_Use_Vx_m_per_sec</code>	0.0	no bulk gas velocity in x
<code>_Use_Vy_m_per_sec</code>	0.0	no bulk gas velocity in y
<code>_Use_Vz_m_per_sec</code>	0.0	no bulk gas velocity in z

The next three field scaling adjustable variables (e.g. `_Use_P_field_scale_factor`) are ignored unless their respective field has been defined (e.g. a non-empty `P_DEFS.DAT` file). These factors and the minimum time step parameter are all briefly discussed below.

Defining Ion Diameter and Reduced Mobility

SDS can estimate values for ion diameter (d nm) and reduced mobility (K_0 10^{-4} $m^2V^{-1}s^{-1}$). However, SDS will also make use of any d and K_0 information that you provide. The `M_DEFS.DAT` file holds d and/or K_0 values organized by ion mass (amu). If you provide d and K_0 values in this file, SDS will automatically associate these values with the ions by matching the mass of the values with mass of the ion. The header section of an `M_DEFS.DAT` file below explains the rules:

```
;holds key diffusion and mobility parameters keyed to mass
;mass (amu), diameter (nm), Ko reduced mobility (10-4 m2 V-1 s-1)
;Note: 1. THERE MUST ALWAYS BE THREE NUMBERS ON EACH LINE (M,D,Ko in that order)
;      190,0.69,1.36 ; legal
;      2. First line with mass = 0 ends list reading (e.g. 0,0.69,1.36)
;      0,0,0 ; legal list termination
;      3. Two lines with same Mass will cause a fatal error
;      190,0.69,1.36 ; legal
;      190,0.85,1.75 ; masses match -- fatal error
;      4. If both d and Ko are 0 --- fatal error
;      190,0,0 ; d = 0 Ko = 0 --- fatal error
;      5. d or Ko can be zero, but not both (be sure to use zeros -- don't skip entry)
;      190,0.69,0 ; d = 0.69 Ko = 0 --- OK
;      190,0,1.36 ; d = 0 Ko = 1.36 --- OK
;      6. Note: If you do not provide a mass def for an ion the user program will
;      will estimate one for you.
;
;      7. Blank lines are allowed
;      8. ; character starts comment: either line or inline
;
;      ; File has two sample mass definitions below
190,0.69,1.36 ; avg positive ion in atmosphere
130,0.61,1.56 ; avg negative ion in atmosphere
```

Thus SDS will use what you provide to estimate what it needs. If values for both K_o and d are provided, they will be used **as is** for each ion defined with a matching value for mass. If only K_o or d are defined, SDS will automatically estimate the undefined (0) value via fitting functions. More information about these methods can be found in the appendix. **If you supply a K_o , SDS assumes it is correct for the buffer gas defined.**

Pressure, Temperature, and Velocity Fields

SDS allows the user to define pressure, temperature, and/or bulk gas velocity fields. These are defined by storing the desired field(s) in the appropriate field definition file(s).

P_DEFS.DAT	Spatial pressure field (empty by default)
T_DEFS.DAT	Spatial temperature field (empty by default)
Vx_DEFS.DAT	Spatial bulk gas x velocity field (empty by default)
Vy_DEFS.DAT	Spatial bulk gas y velocity field (empty by default)
Vz_DEFS.DAT	Spatial bulk gas z velocity field (empty by default)

Each of these files may exist in the current sub-directory. They will not be used if they do not exist or if they are empty (normally they are empty). These data files, if used, **MUST** share the associated potential array dimensions and symmetry (i.e. cylindrical). An example of the blank P_DEFS.DAT file is included to serve as an example. Note: All of these five files have the same required file format:

```
;      Pressure field definition file (torr)
;      Note: Array MUST match its associated potential array
;              ; e.g. same dimensions and symmetry (planar or cylindrical)
;Required file format ;
; nx,ny,nz          ; first three file entries are array dimensions
; val              ; rest of array is linear list of pressure values (torr)
; val              ; (suggest one per line)
; val              ; starting at x,y,z = 0,0,0 and scanning in x, then by y, then by z
; ...
;
; nx = 0            ; signifies an empty array (to be ignored by user program)
; nz = 0            ; 2D cylindrical array
; nz = 1            ; 2D planar array
; nz > 1            ; 3D planar array
;                  ; all non-empty arrays must have same dimensions or fatal error

; Also:            1. Blank lines are allowed
;                  2. ; character starts comment: either line or inline

0,0,0              ; this array is currently declared empty (nx = 0)
```

Thus if the potential array that the user program supports is a 2D cylindrical array with 100 array points in the x direction and 50 array points in the y direction any no-zero data file above that defines an array **MUST** be dimensioned: 100,50,0. The $nz=0$ means 2D cylindrical as described in the file's commented header shown above.

Note: The SDS user program does NOT know the potential array's dimensions. It can only check to make sure all the non-empty field files have the same dimensions and symmetry. It is YOUR responsibility to dimension the data files properly. The penalty for not doing so is garbage out and perhaps a fatal array addressing error.

Note: Data is stored a one dimensional array format. The first data point is assumed to be at 0,0,0 in x,y,z. The second is assumed to be 1,0,0 and so on (x point, by y line, by z page). Thus in a 100x by 50y array the 101st data point will be at 0,1,0 in x,y,z.

Linear interpolation is used to estimate field values via the 4 (2D) or 8 (3D) array corner points that surround the ion's current location. Details concerning the field generating process can be found in the appendix and the two field array subdirectory demos.

Special Issues with Pressure and Temperature fields

Corrections for local pressure and temperature are made by using ratios of local values to the STP values (at the ion's location). In order to avoid division by zero issues, the SDS user program will flag a fatal error if the local temperature or pressure is calculated to be zero. Thus you must preload non-zero values for points in the temperature and/or pressure field arrays. If you get lazy, the SDS will flag a fatal error and tell you that either local pressure or local temperature in the defined field was zero.

If you elect to only define one field (e.g. temperature), the user program will automatically use the user optioned volume wide value for the other (e.g. pressure - `_Use_Pressure_torr`). It is your responsibility to insure that what you include and/or exclude in the way of fields makes physical sense. Garbage in – garbage out!

Special Issues with Bulk Gas Velocity Fields

The Initialize user program segment tests the `VX_DEFS.DAT` file for `nx=0`. If `nx = 0` then the user program assumes that no bulk gas velocity field is defined and the user defined uniform velocity components are assumed (the other two velocity files are ignored).

Unlike pressure and temperature, bulk gas velocity is a vector field. If the potential array is 2D then only the `VX_DEFS.DAT` and `VY_DEFS.DAT` files must be defined, and `VZ_DEFS.DAT` file will be ignored. However, if the potential array is 3D then all three velocity component files must be supplied as non-zero. It is your responsibility to use the potential array's dimensions and symmetry for the field arrays. You can cheat, but you will either get garbage or an array bounds error.

Remember:

$$\nabla \bullet v = 0$$

The most important point here is that it is your responsibility to insure that flow continuity is preserved in the velocity fields you create (e.g. the continuity equation above should hold if compression effects are minimal). Moreover, make sure that flow discontinuities do not occur across any mirrored boundaries. There should be no infinite sheer planes, black holes, or spontaneous mass sources within your flow field. Ignore this at your peril.

A Vignette on the Statistical Diffusion Simulation Method

Viscous flow modeling at elevated pressures, while fast and simple, lacked the capacity to simulate diffusion effects. Diffusion effects are very important if one wants to know what the cloud of ions looks like and how this cloud is impacted by field effects (e.g. refraction and etc.).

It occurred that perhaps one could superimpose a drunkard's diffusion walk upon the viscous trajectories. This might be done by using a statistic of the probability of a radial movement (r_{jump}) as a function the number of expected collisions. The first attempt to create this statistic was greatly simplified. The statistic was generated by assuming each ion started at the origin. It then moved exactly one unit in a random direction to its next collision point. From that collision point it moved exactly one unit in a random direction to its next collision point. Each ion was flown through a simulated number of collisions (say 1000) and its final distance from the starting origin (r_{jump}) was recorded. A large number of ions (e.g. 1,000,000) were flown to obtain a statistic for probability distribution of r_{jump} given a fixed number of collisions.

The comparison of statistics created for different number of collisions (e.g. 10, 100, 1000 and etc.) uncovered the fact that they followed a square root scaling law. That is, the 50% probability r from the 1,000 collision statistic was $\sim 1/10$ th the 50% probability r from the 100,000 collision statistic.

This meant two things. One, a single statistic (using a large number of collisions) could be scaled to represent other statistics with different numbers of expected collisions. Second, the square root scaling would tend to conserve the aggregate r_{jump} accumulated when the expected number of collisions varied (e.g. variable time steps as used in SIMION). This approach looked promising.

An other_actions user program segment was devised to use a defined mean free path, ion time step, and average thermal velocity to estimate the average number of expected collisions in a particular time step. The r_{jump} statistic was loaded in as an array variable and a random number generator was used to pick an r_{jump} value from the distribution. This r_{jump} value from the statistic was then scaled via a square root law:

$$\text{Jump radius: } r_{\text{jump_mm}} = MFP_{\text{ion}} * r_{\text{jump_distribution}} \sqrt{\frac{n_{\text{expected_collisions}}}{n_{\text{distribution_collisions}}}}$$

$$\text{Where: } n_{\text{expected_collisions}} = t_{\text{time_step}} \frac{v_{\text{ion's_average_thermal_speed}}}{MFP_{\text{ion}}}$$

A random direction was chosen and the ion was jumped (instantaneously) a distance of r_{jump} (mm) to its new position. This process was repeated at each time step throughout the ion's trajectory.

Experiments with smaller and larger time steps verified that the diffusion rate of ion trajectories appeared to remain independent of the chosen time step. Next, the physical input parameters of temperature, pressure, ion mass, collision gas mass, ion diameter, and collision gas mass diameter were added to the user program so it could estimate the critical input parameters of K , mean free path, average ion speed.

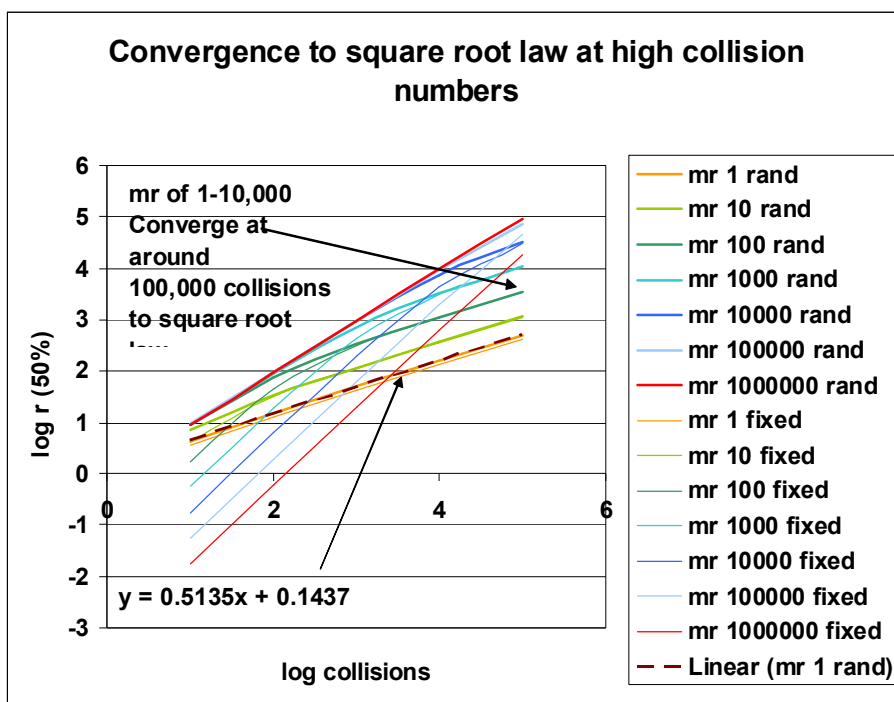
The model predictions were compared to the results obtained from the more classical Monte Carlo collision model (SIMION user program). Initial results indicated that the r_{jump} statistical approach was very promising. The only issue appeared to be an underestimation of the diffusion rates of the outliers. Progressively more representative approaches were used to obtain the r_{jump} statistic. Each successive try expanded the outlier diffusion a bit more.

However, all these approaches assumed that the ion and gas molecules were identical (e.g. the mass ratio = 1). It was decided that the effects of mass ratio must be explored and if possible incorporated into the simulation approach. A series of r_{jump} statistic generating programs were created to model hard sphere impacts in a Monte Carlo manner.

The general observations were: When the mass ratio is one, a single collision can randomly kick an ion off with a new kinetic energy and random direction of travel. However, if the ion is 10,000 times the mass of the bath gas molecules it takes thousands of collisions to totally randomize (de-correlate) its initial velocity. Thus a heavy ion tends to retain its initial velocity largely unchanged through many collisions (linear r scaling region). However, as the collisions mount up it de-correlates to the point that r begins to scale as the square root as described above (desirable).

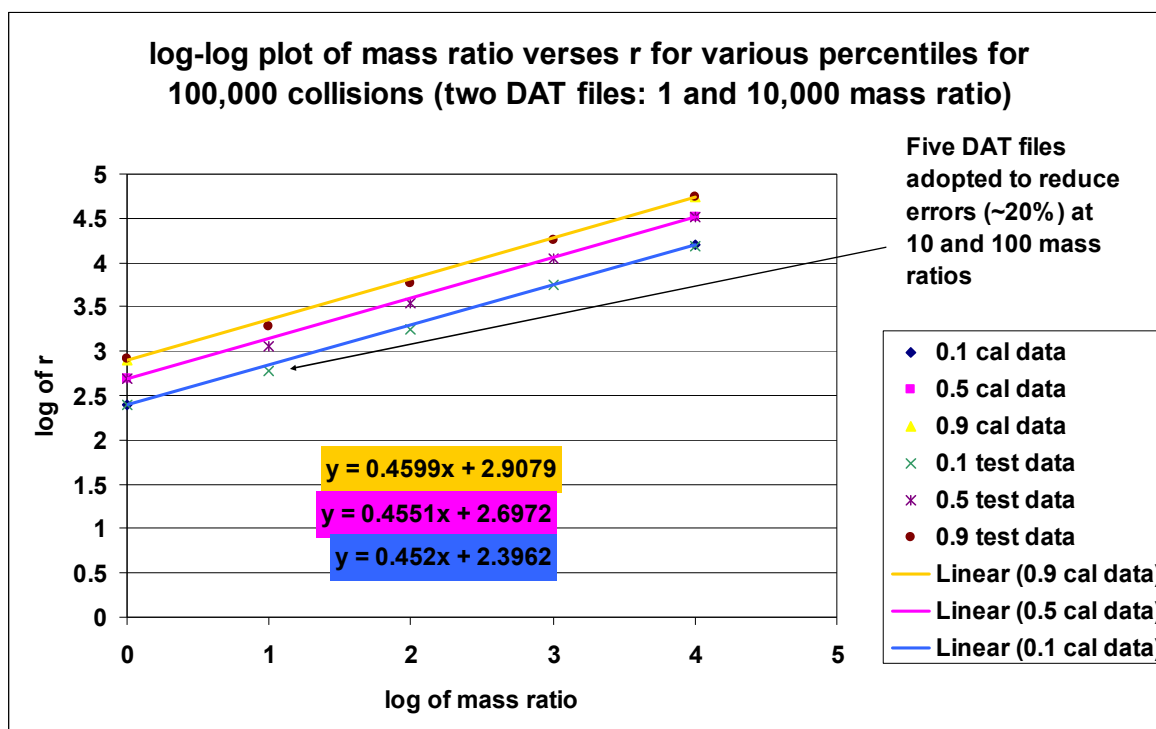
Two collections of simulations were run in order to obtain a better understanding of the process. The first collection of simulations, referred to as fixed, always initialized each ion with a velocity of 1 (normalized) along the x axis, and always used a fixed distance step of 1 (normalized). The second collection of simulations started each ion out with a random Maxwell Boltzman energy (avg. normalized to 1) and direction, and each distance step was Poisson randomized to an average of 1 (normalized). Each simulation flew 10,000 ions. A separate simulation was run for each of five different total number of collisions (10, 100, 1000, 10000, or 100000), and had one of seven mass ratios (1, 10, 100, 1000, 10000, 100000, or 1000000). Thus each collection had 35 simulations.

The plot below shows a comparison for the variation of the 50% r distribution point for these simulations. The most important observation is that both collections with the equivalent mass ratios converge to common r (50%) values when the number of collisions is large enough. This means that both approaches lead to the same r statistics for large values of n collisions. Also notice that the lower mass ratios converge at a lower number of n collisions.



This graph lead to the insight that if the number of collisions are high enough, $\sim 100,000$, mass ratios of less than 10,000 to 1 will have converged into the square root law. This means that if the time steps are large enough for 100,000 collisions we can ignore the linear rule region and assume the ion trajectories will be fully de-correlated and fly in the square root law scaling region. This condition is easily satisfied for ions flying in atmospheric conditions using the sub to one mm integration steps typically used by SIMION.

The question remained as how to scale between mass ratios. Initial evidence indicated that two ion simulations of 100,000 collisions per ion and mass ratios 1 and 10,000 might scale linearly on a log-log plot. A more detailed check shows (as shown in the plot below) that mass ratios of 10 and 100 deviate by 20% or so from the simple linear assumption. In order to reduce these errors five 100,000 ion by 100,000 collisions per ion simulations (at 1, 10, 100, 1000, 10000 mass ratios) were run to create the master set of scalable statistics.



The interpolation scheme used involves selecting the distribution percentage by a random number, getting the $\log_{10}$ of the collision mass ratio (mr), use it to select the two adjacent statistics:

Assume: $mr = 25$:
 Then: $\log(mr) = 1.39$:
 Therefore: use MBMR1 ($mr = 10$) and MBMR2 ($mr = 100$)

Next the r value from each statistic is found by using the random number percentage selection process above:

$$\begin{aligned} \text{index} &= 1,000 * \text{random_number}(0-1) \\ r_{1\text{jump}} &= \text{MBMR1} [\text{index}] \\ r_{2\text{jump}} &= \text{MBMR2} [\text{index}] \end{aligned}$$

The $\log_{10}$ of each r value is taken and the $\log_{10}$ r value at the desired mass ratio is obtained by linear (log-log) interpolation. The result is raised to the 10^{th} power to get r_{obtained} .

$$\begin{aligned} \log(r_{\text{jump_obtained}}) &= \log(r_{1\text{jump}}) + 0.39 * (\log(r_{2\text{jump}}) - \log(r_{1\text{jump}})) \\ r_{\text{jump_obtained}} &= 10^{\log(r_{\text{jump_obtained}})} \end{aligned}$$

This r value is then square root scaled to the expected number of collisions:

Jump radius:
$$r_{jump_expected} = MFP_{ion} * r_{jump_obtained} \sqrt{\frac{n_{expected_collisions}}{n_{distribution_collisions}}}$$

Where:
$$n_{expected_collisions} = t_{time_step} \frac{V_{ion's_average_thermal_speed}}{MFP_{ion}}$$

Details of the Assumptions and Methods Used in SDS

The following discussion details the internal workings of the SDS user program segments. There are four user program segments involved:

1. The initialize program segment is used to initialize the process by either getting information from the user or by making its best estimate of the key operating parameters (e.g. using the ion's mass as a starting point).
2. The other_actions program segment is used to randomly jump the ion to simulate diffusion by hard sphere collisions (using the statistics and method above).
3. The accel_Adjust program segment uses Stoke's law to simulate the K mobility and any bulk movement (e.g. $V_{bulk\ gas}$) of the buffer gas.
4. The tstep_adjust program segment allows the user to dynamically adjust the minimum allowed time step (by default zero) to force faster ion group flying in sharply changing voltage gradient regions.

Structural considerations due to V_{local} , T_{local} and P_{local} fields

Each ion has its own flight parameters (e.g. damping factor (mobility based), average thermal velocity, and mean free path) allowing simulations of mixed groups of ions to be conducted. This is accomplished by maintaining arrays that hold the individual key parameters needed for each ion (indexed to ion_number). The key ion parameters stored in arrays are as follows (in SIMION used units):

1. $\text{Log}_{10}(\text{mass}_{ion}/\text{mass}_{gas})$
2. $k_{damping_0}$ (at STP) (1/μsec)
3. MFP_0 mean free path (at STP) (mm)
4. V_0 average thermal speed (at STP) (mm/μsec)
5. $k_{local_damping}$ (at T and P local to ion) (1/μsec)
6. MFP_{local} (at T and P local to ion) (mm)
7. V_{local} (at T and P local to ion) (mm/μsec)

In addition to the above arrays there are five local variables used. Two are used to hold the local temperature and pressure and three for the bulk gas 3d rectangular velocity components at the current ion's location:

local_pressure_torr
local_temperature_K
local_vx_mm_per_usec
local_vy_mm_per_usec
local_vz_mm_per_usec

These local variables are controlled in the following manner. When SDS the user program initializes, these local variables are initially set to the user specified constant values (using appropriate scaling) from the following variables:

`_Use_Pressure_torr`
`_Use_Temperature_K`
`_Use_Vx_m_per_sec`
`_Use_Vy_m_per_sec`
`_Use_Vz_m_per_sec`

The SDS user program also can use spatially varying fields of pressure, temperature, and bulk gas velocity. These field definition files will not be used if they do not exist in the current sub-directory or if they are empty. Fields must match the associated potential array perfectly (e.g. in dimensions and symmetry (i.e. cylindrical)). The user passes these field definitions to the user program via five data files. They can be empty (their default condition or marked as empty by setting each file's nx dimension to zero). Therefore, while these files may exist, they are generally ignored by the user program unless the user has placed data in them and made them active:

Holds pressure field data	<code>P_DEFS.DAT</code>
Holds temperature field data	<code>T_DEFS.DAT</code>
Holds Vx bulk gas velocity data	<code>VX_DEFS.DAT</code>
Holds Vy bulk gas velocity data	<code>VY_DEFS.DAT</code>
Holds Vz bulk gas velocity data	<code>VZ_DEFS.DAT</code>

These arrays are normally empty or flagged as empty ($n_x = 0$). Thus the local values of temperature, pressure, and bulk gas velocity are generally set to the fixed constant values passed to the user program by the user via the use variables described above.

The initialize program segment's job is to initialize all 7 ion specific data arrays properly for use by the `accel_adjust` and `other_actions` user program segments. The discussion below generally assumes that the temperature, pressure, and bulk gas velocity are all constant valued fields.

The Initialize User Program Segment

Most of the computational setup for SDS is performed by the initialize program segment. The key parameters required by the `accel_adjust` and the `other_actions` user program segments are: $\log_{10}(\text{mass}_{\text{ion}}/\text{mass}_{\text{gas}})$, $k_{\text{local_damping}}$, V_{local} , $\text{MFP}_{\text{local}}$, $V_{x\text{ local}}$, $V_{y\text{ local}}$, and $V_{z\text{ local}}$ (local means adjusted for the values of temperature, pressure, and velocity at the ion's current position).

During the initialization process standard temperature and pressure (STP) values for viscous damping (k_{damping0}), mean free path (MFP_0), and average thermal speed (V_0) are calculated for each ion and stored within the appropriate array (array lines 2-4 above). During this process the user supplied local fixed values for pressure, temperature, and bulk gas velocity, are used to initialize the local $k_{\text{local_damping}}$, $\text{MPF}_{\text{local}}$, and V_{local} arrays

properly (array lines 5-7 above) for each ion as well as the local bulk gas velocity vector component variables.

The initialize program segment checks to see if one or more of the five field defining data files below is non-empty (nx != 0):

Holds pressure field data	P_DEFS.DAT
Holds temperature field data	T_DEFS.DAT
Holds Vx bulk gas velocity data	VX_DEFS.DAT
Holds Vy bulk gas velocity data	VY_DEFS.DAT
Holds Vz bulk gas velocity data	VZ_DEFS.DAT

It tests the all non-empty files to verify that they all have the same dimensions and symmetry. If all is ok, a flag will be set to allow the accel_adjust user program segment to call its special subroutine that updates local field values from the defined field array values as described above.

Methods used to obtain/estimate d and K_o

The STP (or reduced) mobility K_o could obtained via the following formula:

$$K_o = \frac{3e}{16N_o} \left(\frac{2\pi}{\mu kT} \right)^{1/2} \frac{1}{\Omega}$$

Though it is satisfying to obtain mobility estimates from first principles, the complex effects controlling ion mobility are more than a little disquieting. All sorts of effects from dipole moments to agglomeration appear to play an active role in real world ion mobility measurements. This tends to make the estimation of observed ion mobility from first principles fraught with peril. Measuring K is often more straightforward than deriving or predicting it.

In view of these sobering revelations about K and, to some extent, ion diameters (d_{ion}), it was decided to provide a method by which the user could specify what was known: d and/or K_o or neither. The SDS user program would make use of what the user defined (in the M_DEFS.DAT file) or estimate the values as best it could from what was supplied.

The M_DEFS.DAT file

The SDS user program requires the existence of the file M_DEFS.DAT in the current directory. This file holds any ion mobility (K_o) and ion diameter (d) information supplied by the user. The data is organized according to mass. Ions that have mass that match the mass of a given definition line in the table will be given the K_o and d attributes defined (or SDS estimated). The following is an example of this file with two mass entries:

SDS User Program Documentation

```
;holds key diffusion and mobility parameters keyed to mass
;mass (amu), diameter (nm), Ko reduced mobility (10-4 m2 V-1 s-1)
;Note: 1. THERE MUST ALWAYS BE THREE NUMBERS ON EACH LINE (M,D,Ko in that order)
;      190,0.69,1.36 ; legal
;      2. First line with mass = 0 ends list reading (e.g. 0,0.69,1.36)
;      0,0,0          ; legal list termination
;      3. Two lines with same Mass will cause a fatal error
;      190,0.69,1.36 ; legal
;      190,0.85,1.75 ; masses match -- fatal error
;      4. If both d and Ko are 0 --- fatal error
;      190,0,0        ; d = 0 Ko = 0 --- fatal error
;      5. d or Ko can be zero, but not both (be sure to use zeros -- don't skip entry)
;      190,0.69,0      ; d = 0.69 Ko = 0 --- OK
;      190,0,1.36      ; d = 0 Ko = 1.36 --- OK
;      6. Note: If you do not provide a mass def for an ion the user program will
;      will estimate one for you.
;      7. Blank lines are allowed
;
;      8. ; character starts comment: either line or inline
;
;      ; File has two sample mass definitions below
190,0.69,1.36      ; avg positive ion in atmosphere
130,0.61,1.56      ; avg negative ion in atmosphere
```

These are the rules used for this file:

1. Every data line must contain THREE numbers: mass, diameter, and reduced mobility (in that order).
2. A non-zero value for mass must be provided or the list reading will stop at that line in the file.
3. The SDS user program uses the mass value to associate the specified parameters with any ion that has the same mass value (be careful).
4. If both d and Ko are defined, they will always be used, as defined, for Ko and for calculating the ion's STP collision frequency.
5. Either d or Ko can be replaced by a zero (0). The user program will then estimate one from the other as describe below.
6. If both d and Ko are zero (0) in any data line with a non-zero mass, a fatal error will be processed and the user will be notified of the cause of the error.
7. If the same non-zero value for mass is defined on more than on data line, a fatal error will be processed and the user will be notified of the cause of the error.
8. If the file is empty (no non-zero mass data lines defined) or a matching mass can't be found for an ion's mass, the user program will assume nothing is known about either d or K for ion's mass. It will then estimate both d and Ko by using the ion's mass as a starting point as described below (and add it to the internal table).

D and K_o estimation methods

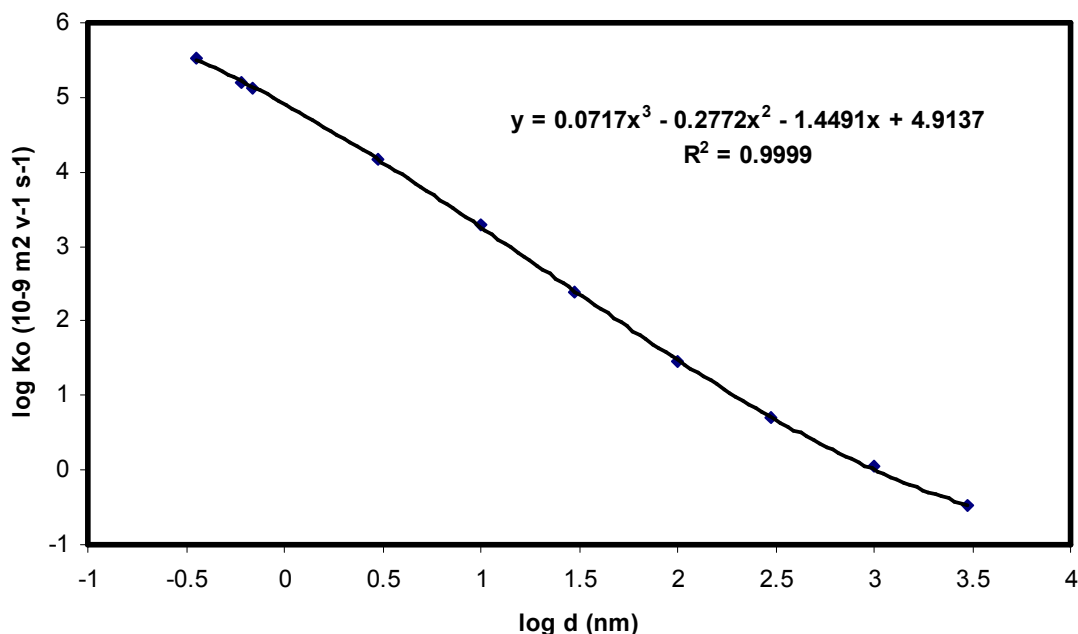
If no mass is supplied in M_DEFS.DAT that matches the ion's mass, the user program will assume the neither d nor K_o are known. It will first estimate the value of d from the mass of the ion via the formula below (atmospheric aerosols 14-35 Handbook of Chemistry and Physics 83rd edition: -ions, avg mass = 130 amu, equiv diameter 0.61 nm used to obtain mass to d scaling constant – **just a simple assumption of constant volume density**):

$$d_{ion}(nm) = 0.120415405(mass_{ion}(amu))^{1/3}$$

Estimation of K_o

If K_o is not provided, it will be estimated from either the d_{ion} value supplied in the M_DEFS.DAT file or the d_{ion} value obtained from the above formula (if no matching mass value is found in the file). The formula used was derived by fitting log-log plots of ion diameter and K_o values in air (from page 14-36 The Handbook of Chemistry and Physics 83rd edition) to a third order polynomial (*corrections for buffer gas follow*):

Mobility as a Function of Ion Diameter



	diameter d_{ion} (nm)	Mobility K_o (10 ⁻⁹ m ² V ⁻¹ s ⁻¹)	Mobility from d to K_o fit
O ₂	0.355	328000	318320.3
-ions	0.61	156000	162665.3
+ions	0.69	136000	137951
From	3	15000	14688.86
Table	10	1900	1815.934
	30	250	250.8807
	100	28	30.22039
	300	5.1	5.164211
	1000	1.11	1.017419
	3000	0.33	0.344983

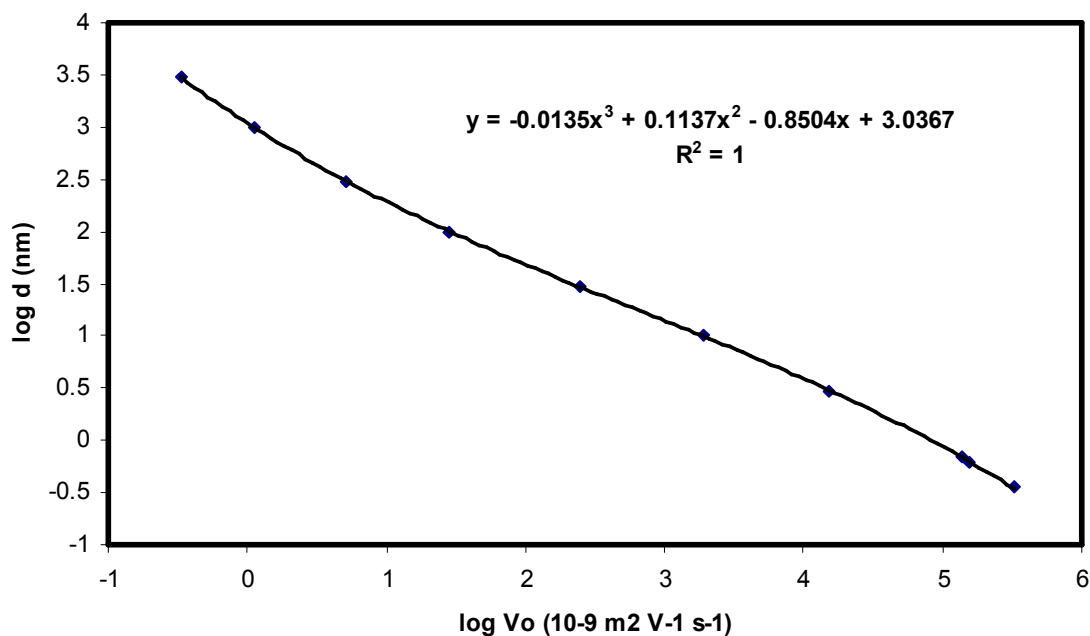
$$K_o(10^{-4} m^2 V^{-1} s^{-1}) = 1.0 \times 10^{-5} * 10^{(A)}$$

$$A = 4.9137 - 1.4491(\log_{10}(d(nm))) - 0.2772(\log_{10}(d(nm)))^2 + 0.0717(\log_{10}(d(nm)))^3$$

Estimation of d_{ion}

If d_{ion} is not provided, it will be estimated either from the K_o value supplied in the M_DEFS.DAT file or the from the ion's mass value formula above if no matching mass value is found in the file. The K_o to d_{ion} formula below was derived by fitting log-log plots of ion diameter and K_o values in air (from page 14-36 The Handbook of Chemistry and Physics 83rd edition) to a third order polynomial (corrections for buffer gas follow):

Ion Diameter as a Function of Mobility



	Mobility Ko (10-9 m2 V-1 s-1)	diameter d_{ion} (nm)	Ko to d fit d_{ion} (nm)
O ₂	328000	0.355	0.346531
-ions	156000	0.61	0.625319
+ions	136000	0.69	0.693663
From	15000	3	3.055245
Table	1900	10	9.88347
	250	30	29.18342
	28	100	100.7488
	5.1	300	306.9902
	1.11	1000	996.2989
	0.33	3000	2978.664

$$d_{ion}(nm) = 10^{(A)}$$

$$A = 3.0367 - 0.8504(\log_{10}(K_9)) + 0.1137(\log_{10}(K_9))^2 - 0.0135(\log_{10}(K_9))^3$$

Where: $K_9(10^{-9} m^2 V^{-1} s^{-1}) = 1.0 \times 10^5 K_o(10^{-4} m^2 V^{-1} s^{-1})$

Correcting K_o and d_{ion} estimates for buffer gas used

The estimating process described above assumes air is the buffer gas (mass = 28.954515 amu and $d = 0.366$ nm – the SDS default). If the buffer gas is not air then the estimates will be corrected. **Note: If you defined both K_o and d_{ion} for the ion's mass in the M_DEFS.DAT file, the SDS program will assume that they are the correct values to use with the buffer gas you defined and no corrections will be applied.**

From gas kinetics:

$$K_o = \frac{3e}{16N_o} \left(\frac{2\pi}{\mu kT} \right)^{1/2} \frac{1}{\Omega}$$

Where:

$$\mu = reduced_mass = \frac{mass_{ion} * mass_{gas}}{mass_{ion} + mass_{gas}}$$

And:

$$\Omega = cross_section \approx \pi \left[\frac{d_{ion} + d_{gas}}{2} \right]^2$$

Thus K_{ogas} and K_{oair} can be estimated from each other as follows:

$$K_{ogas} = K_{oair} * C_{gas/air}$$

$$K_{oair} = \frac{K_{ogas}}{C_{gas/air}}$$

Where:

$$C_{gas/air} = \left[\frac{\mu_{air}}{\mu_{gas}} \right]^{1/2} \left[\frac{\Omega_{air}}{\Omega_{gas}} \right]$$

$$C_{gas/air} = \left[\frac{\left(\frac{mass_{ion} * mass_{air}}{mass_{ion} + mass_{air}} \right)}{\left(\frac{mass_{ion} * mass_{gas}}{mass_{ion} + mass_{gas}} \right)} \right]^{1/2} \left[\frac{d_{ion} + d_{air}}{d_{ion} + d_{gas}} \right]^2$$

When d_{ion} is known (or estimated) but K_{ogas} is unknown, K_{oair} is first calculated using the fitting equations described above and then multiplied by $C_{air/gas}$ to estimate K_{ogas} for the buffer gas.

When K_{ogas} is specified but d_{ion} is unknown a different approach is used. We need an estimate of K_{oair} to use the fitting function to estimate d_{ion} . This requires $C_{gas/air}$. However, $C_{gas/air}$ requires knowing d_{ion} . To simplify the calculation an initial estimate of d_{ion} is obtained using the mass of the ion (as described above). This estimate is then used to calculate $C_{gas/air}$. K_{ogas} is divided by $C_{gas/air}$ to obtain an estimate of K_{oair} . K_{oair} is then used with the fitting function to obtain the final estimate of d_{ion} .

The other_actions User Program Segment

The other_actions user program segment is used to simulate diffusion effects via a collection of collision statistics described above. The primary inputs required by the statistics are the $\log_{10}$ mass ratio and the local temperature and local pressure corrected average thermal speed and mean free path of each ion. This section will discuss the methods used to obtain each ion's average thermal speed and mean free path.

As stated in the previous section, these results are stored for each ion in specific arrays. Normally, the values are determined by the initialize user program segment by using the constant pressure and temperatures defined by the user. However, when temperature and/or pressure fields are defined (via non-empty file(s): P_DEFS.DAT and/or T_DEFS.DAT), a subroutine within the accel_adjust program segment is automatically called to obtain the correct local field value(s) for the ion's local temperature and/or pressure. These values are then used to obtain local values for damping constant, mean free path, and ion average speed these values are stored in the appropriate ion arrays (lines 5-7 above).

Obtaining the average thermal speed of each ion

The average thermal speed of each ion is determined using the formula for the average thermal speed of the Maxwell Boltzman distribution:

$$\text{Average ion speed at STP} \quad V_o = \sqrt{\frac{8kT_o}{\pi m}} (ms^{-1})$$

Where:

$$T_o = 273.15^\circ K$$

k is the Boltzman constant

$$k = 1.3806503 \times 10^{-23} (JK^{-1})$$

m is the mass of the ion

$$m = \text{mass (kg)}$$

$$m \text{ (amu)} \quad V_o = \sqrt{\frac{8 * 1.3806503 \times 10^{-23} * 273.15}{1.66053873 \times 10^{-27} (kg / amu) \pi}} \sqrt{\frac{1}{m(am u)}} (ms^{-1})$$

$$V_o = 2.404850043 \times 10^3 \sqrt{\frac{1}{m(am u)}} (ms^{-1})$$

$$\text{In SIMION units:} \quad V_o = 2.404850043 \sqrt{\frac{1}{m(am u)}} (mm \mu sec^{-1})$$

For V at local temperature:

$$V = V_o \sqrt{\frac{T_{local}}{T_o}}$$

Temperature scaling used:

$$V = V_o \sqrt{\frac{T_{local}}{273.15}}$$

Method used to Determine Mean Free Path

The mean free path of an ion can be considered in two different ways. The first way is to assume that the ion is flying through a cloud of relatively stationary molecules (targets). The number of collisions per distance flown relates to the volume of the path swept out and the density of the targets within that volume. The collision rate for this simple assumption is defined by the following formula:

$$\text{Collision rate (1/sec):} \quad \nu_{rate} = \pi \left(\frac{d_{ion} + d_{gas}}{2} \right)^2 N \bar{v}_{ion}$$

$$\nu_{rate} = \text{collisions / sec}$$

$$d_{ion} = \text{diameter}_{ion} (m)$$

$$\text{Where:} \quad d_{gas} = \text{diamenter}_{gas} (m)$$

$$N = \text{density}_{gas} (n / m^3)$$

$$\bar{v}_{ion} = \text{average_speed}_{ion} (m / \text{sec})$$

When the motions of the buffer gas are taken into consideration and the buffer gas molecules are roughly the same as the ions the collision frequency can be shown to increase by a factor of the square root of 2:

$$\text{Collision rate (1/sec):} \quad \nu_{rate} = \pi \sqrt{2} \left(\frac{d_{ion} + d_{gas}}{2} \right)^2 N \bar{v}_{ion}$$

The mean free path between collisions in is found by dividing the average thermal velocity of the ion by its collision frequency (somewhat roundabout at this point):

$$\text{Mean free path (m)} \quad MFP = \frac{\bar{v}_{ion}}{\nu_{rate}} \left(\frac{m / \text{sec}}{1 / \text{sec}} \right)$$

The second way mean free path can be considered is by assuming that the ion is very large (slow moving) and the collision gas moves very rapidly. Thus the expected collision frequency is based on multiplying the surface area of the ion times the collision frequency per stationary unit area of the gas molecules.

$$\text{Collision rate (1/sec):} \quad \nu_{rate} = \pi (d_{ion})^2 \frac{N \bar{v}_{gas}}{4}$$

$$\nu_{rate} = \text{collisions / sec}$$

$$d_{ion} = \text{diameter}_{ion} (m)$$

$$\text{Where:} \quad N = \text{density}_{gas} (n / m^3)$$

$$\bar{v}_{gas} = \text{average_speed}_{gas} (m / \text{sec})$$

The issue here is that the size of the ion will be specified by the user. This means that the collision rate will transition between the first and second formulas as the ion increases in size (and mass) relative to the bath gas. An early version of this user program calculated both collision frequencies and picked the one that resulted in the shortest mean free path (not too satisfying). The approach adopted for the SDS user program is to create a formula that combines these two collision frequencies in one formula to smoothly transition the frequency between modes as the ion size (mass) increases:

Combined:
$$\nu_{rate} = 1.164213562 \pi \left(\frac{d_{ion} + d_{gas}}{2} \right)^2 N \bar{v}_{ion} + \pi (d_{ion})^2 \frac{N \bar{v}_{gas}}{4}$$

Where:
$$1.164213562 = \sqrt{2} - \frac{1}{4}$$

Thus the composite formula becomes the first formula when the ion and gas molecules are the same. However as the ion gets more massive the second formula gradually dominates as the collision frequency.

The following is a combination of the terms to create the formula for the expected collision rate at STP (standard temperature and pressure).

At STP:
$$\nu_{rateo} = 1.164213562 \pi \left(\frac{d_{ion} + d_{gas}}{2} \right)^2 N_o \bar{v}_{iono} + \pi (d_{ion})^2 \frac{N_o \bar{v}_{gaso}}{4}$$

$$\nu_{rateo} = \text{collisions} / \text{sec}(\text{STP})$$

$$d_{ion} = \text{diameter}_{ion} (m)$$

$$d_{gas} = \text{diameter}_{gas} (m)$$

Where:

$$N_o = \text{density}_{gas} (n / m^3)(\text{STP})$$

$$\bar{v}_{iono} = \text{average_speed}_{ion} (m / \text{sec})(\text{STP})$$

$$\bar{v}_{gaso} = \text{average_speed}_{gas} (m / \text{sec})(\text{STP})$$

Simplifying:
$$\nu_{rateo} = \pi N_o \left(1.164213562 \left(\frac{d_{ion} + d_{gas}}{2} \right)^2 \bar{v}_{iono} + (d_{ion})^2 \frac{\bar{v}_{gaso}}{4} \right)$$

Converting to units used in SIMION:

$$\nu_{rateo} = A_o \left(1.164213562 \left(\frac{d_{ion} + d_{gas}}{2} \right)^2 \bar{v}_{iono} + (d_{ion})^2 \frac{\bar{v}_{gaso}}{4} \right)$$

$$A_o = 1.0 \times 10^{-12} (mm^2 / nm^2) 1.0 \times 10^{-9} (m^3 / mm^3) \pi N_o$$

And:
$$A_o = (1.0 \times 10^{-21}) (3.141592654) \frac{6.02214199 \times 10^{23} (mol^{-1})}{22.413996 (m^3 mol^{-1})}$$

$$A_o = 8.44076042 \times 10^1$$

Where:

$$v_{rateo} = collisions / \mu sec (STP)$$

$$d_{ion} = diameter_{ion} (nm)$$

$$d_{gas} = diameter_{gas} (nm)$$

$$N_o = density_{gas} (n / m^3) (STP)$$

$$\bar{v}_{iono} = average_speed_{ion} (mm / \mu sec) (STP)$$

$$\bar{v}_{gaso} = average_speed_{gas} (mm / \mu sec) (STP)$$

The estimated mean free path (MFP_o) of the ion at STP (in SIMION units) is:

$$MFP_o (mm) = \frac{\bar{v}_{iono} (mm / \mu sec)}{v_{rateo} (collisions / \mu sec)}$$

Local Mean Free Path (MFP_{local})

Mean free path can be adjusted for local temperature and pressure at the ion's location by the following formula:

Since:
$$MFP \propto \left(\frac{1}{d^2} \right) \left(\frac{T}{P} \right)$$

Then:
$$MFP_{local} = MFP_o \left(\frac{T_{local}}{T_o} \right) \left(\frac{P_o}{P_{local}} \right)$$

Values used:
$$MFP_{local} = MFP_o \left(\frac{T_{local}}{273.15(^{\circ} K)} \right) \left(\frac{760(Torr)}{P_{local}} \right)$$

The accel_adjust User Program Segment

This user program segment applies mobility based viscous effects to the ion motions. Support for bulk gas velocities is also supported. A subroutine within this user program segment is used to get local pressure, temperature, and/or bulk gas velocity from any user defined fields. Computational methods are presented below:

Converting *K* mobility to damping constant

If viscous damping is assumed, the following equation for acceleration holds for acceleration from rest within a fluid of constant velocity:

Acceleration decreases exponentially:
$$A = A_o e^{-k_o t}$$

Velocity is obtained by integration: $V = \int_0^t A_o e^{-k_o t} dt$

Velocity increases to fixed value: $V = \frac{A_o}{k} (1 - e^{-k_o t})$

Maximum velocity is V_o :

$$V_o = \frac{A_o}{k_o}$$

$$k_o = A_o / V_o$$

Thus the time constant k is the ratio of the initial acceleration A_o and the maximum velocity V_o .

Ion mobility is defined by the following relation:

Maximum velocity: $V_o = K_o \frac{Volts}{m}$

Where K is ion mobility ($m^2 Volts^{-1} sec^{-1}$)

Given: Work = force * distance
 Then: force = work / distance
 Since: force = mass * acceleration
 Then: acceleration = force / mass

Thus: $A_o = \frac{Force}{mass} = \frac{Joule}{mass * distance}$

Given: Joule = volts / coulomb

A_o can be defined in the units: $A_o = \frac{e * Volts}{mass * m}$

Given $k_o = A_o / V_o$

Substituting: $k_o = \frac{\frac{e * volts}{mass * m}}{\frac{K_o * volts}{m}} = \frac{e(coulombs)}{mass(kg) * K_o(m^2 volts^{-1} sec^{-1})}$

Since SIMION works with elementary charges and mass is defined in amu (m_u), the following conversion constant is required:

$$e/m_u = 1.602176462 \times 10^{-19} / 1.66053873 \times 10^{-27}$$

$$e/m_u = 9.648434136 \times 10^7$$

Thus k (1/sec):
$$k_o = \frac{9.648434136 \times 10^7}{\text{mass}(\text{amu}) * K_o (\text{m}^2 \text{volts}^{-1} \text{sec}^{-1})}$$

And k (1/μsec):
$$k_o = \frac{9.648434136 \times 10^1}{\text{mass}(\text{amu}) * K_o (\text{m}^2 \text{volts}^{-1} \text{sec}^{-1})}$$

The traditional units for K_o are: (cm²Volts⁻¹sec⁻¹) or (10⁻⁴ m²Volts⁻¹sec⁻¹)

Thus:
$$K_o (\text{m}^2 \text{Volts}^{-1} \text{sec}^{-1}) = 0.0001 * K_o (10^{-4} \text{ m}^2 \text{Volts}^{-1} \text{sec}^{-1})$$

Thus k (1/sec):
$$k_o = \frac{9.648434136 \times 10^{11}}{\text{mass}(\text{amu}) * K_o (10^{-4} \text{ m}^2 \text{volts}^{-1} \text{sec}^{-1})}$$

And k (1/μsec) * :
$$k_o = \frac{9.648434136 \times 10^5}{\text{mass}(\text{amu}) * K_o (10^{-4} \text{ m}^2 \text{volts}^{-1} \text{sec}^{-1})}$$

***The equation above is the one used by the initialize program segment.**

Converting the STP k_o to local k Value

Any K value input to the SDS user program **by the user** is assumed to be the reduced value of K_o (10⁻⁴ m²volts⁻¹sec⁻¹). The initialize user program segment then uses the K_o value defined (or estimated) and combines this with the ion's mass to calculate the initial entry for the k_o constant at STP for each ion in a k_o array (using the formula above). The local values for temperature and pressure are then used (via the formula below) to obtain the local k value which is stored in a local k_{local_damping} array.

Given:
$$k_o \propto \frac{1}{K_o}$$

Since:
$$K = K_o \frac{P_o}{P} \frac{T}{T_o}$$

Then:
$$k_{\text{local_damping}} = k_o \left(\frac{P_{\text{local}}}{P_o} \right) \left(\frac{T_o}{T_{\text{local}}} \right)$$

Bulk gas motions

The accel_adjust user program segment also includes the effects of the local bulk gas velocity. Local 3D rectangular coordinate values of the bulk gas velocity are subtracted from the damping velocity before the acceleration due to damping is calculated. These values are normally constant velocity values (user initialized).

$$\begin{aligned} V_{x \text{ damping}} &= V_{x \text{ ion}} - V_{x \text{ local bulk gas}} \\ V_{y \text{ damping}} &= V_{y \text{ ion}} - V_{y \text{ local bulk gas}} \\ V_{z \text{ damping}} &= V_{z \text{ ion}} - V_{z \text{ local bulk gas}} \end{aligned}$$

If bulk gas velocity fields are defined (e.g. non-empty data files VX_DEFS.DAT and VY_DEFS.DAT and perhaps VZ_DEFS.DAT (3d potential arrays only)), the accel_adjust program segment will also update the local velocity variables (above) using the velocity field information supplied. Details about velocity field definition and calculation are described below.

Defining Pressure, Temperature, and Bulk gas Velocity Fields

The SDS user program allows the user to define pressure, temperature and bulk gas velocity fields via five specifically named files (alluded to above):

Holds pressure field data	P_DEFS.DAT
Holds temperature field data	T_DEFS.DAT
Holds Vx bulk gas velocity data	VX_DEFS.DAT
Holds Vy bulk gas velocity data	VY_DEFS.DAT
Holds Vz bulk gas velocity data	VZ_DEFS.DAT

Each of these files may exist in the current sub-directory. They will not be used if they do not exist or if they are empty (normally they are empty). These data files, if used, **MUST** share the associated potential array dimensions and symmetry (i.e. cylindrical). An example of the blank P_DEFS.DAT file is included to serve as an example. Note: All of these five files have the same required file format:

```
;      Pressure field definition file (torr)
;      Note: Array MUST match its associated potential array
;              ; e.g. same dimensions and symmetry (planar or cylindrical)
;Required file format ;
; nx,ny,nz          ; first three file entries are array dimensions
; val               ; rest of array is linear list of pressure values (torr)
; val               ; (suggest one per line)
; val               ; starting at x,y,z = 0,0,0 and scanning in x, then by y, then by z
; ...
;
; nx = 0            ; signifies an empty array (to be ignored by user program)
; nz = 0            ; 2D cylindrical array
; nz = 1            ; 2D planar array
; nz > 1            ; 3D planar array
;                  ; all non-empty arrays must have same dimensions or fatal error

; Also:            1. Blank lines are allowed
;                  2. ; character starts comment: either line or inline

0,0,0              ; this array is currently declared empty (nx = 0)
```

Thus if the potential array that the SDS user program supports is a 2D cylindrical array with 100 array points in the x direction and 50 array points in the y direction any no-zero data file above that defines an array **MUST** be dimensioned: 100,50,0. The nz=0 means 2D cylindrical as described in the file's commented header shown above.

Note: The SDS user program does NOT know the potential array's dimensions. It can only check to make sure all the non-empty field files have the same dimensions and symmetry. It is YOUR responsibility to dimension the data files properly. The penalty for not doing so is garbage out and perhaps a fatal array addressing error.

Note: All data is stored a one dimensional array format. The first data point is assumed to be at 0,0,0 in x,y,z. The second is assumed to be 1,0,0 and so on (x point, by y line, by z page). Thus in a 100x by 50y array the 101st data point will be at 0,1,0 in x,y,z.

How the local value is obtained

The SDS user program employs linear interpolation to obtain the estimate for the P, T, Vx, Vy, and/or Vz at the ion's current location in the potential array. Let's say P_DEFS.DAT is defined for a 3D array of nx = 20, ny = 30, nz = 40 and the ion is currently at 10.3x, 12.6y, 5.9z in absolute potential array coordinates (e.g. ion_px_abs_gu, ion_py_abs_gu, and ion_pz_abs_gu). The ion is currently inside the eight array point 3D box defined by the 3D diagonal array points (10,12,5) and (11,13,6). The horizontal weights are calculated as follows assuming the lower left corner has a relative offset of 0,0,0 in x, y, and z:

Note: The user program only uses 4 points with 2D arrays.

Array dimensions:	Nx = 20 Ny = 30 Nz = 40		
Offset of llc point:	Offset = x + (y * nx) + (z * nx * ny) Offset = 10 + (12 * 20) + (5 * 20 * 30) Offset = 3,250		
Point's dx, dy, and dx interpolation weighting factors	X weight fx = 0.3 Y weight fy = 0.6 Z weight fz = 0.9	(1 - fx) = 0.7 (1 - fy) = 0.4 (1 - fz) = 0.1	
Lower page	Rel. Coord.	Point rel. array offset	Point's total weighting factor
llc	(0,0,0)	0	(1 - fx)(1 - fy)(1 - fz)
lrc	(1,0,0)	1 (+1)	(fx)(1 - fy)(1 - fz)
ulc	(0,1,0)	20 (nx)	(1 - fx)(fy)(1 - fz)
urc	(1,1,0)	21 (nx+1)	(fx)(fy)(1 - fz)
Upper z page			
llc	(0,0,1)	600 (nx*ny)	(1 - fx)(1 - fy)(fz)
lrc	(1,0,1)	601 (nx*ny + 1)	(fx)(1 - fy)(fz)
ulc	(0,1,1)	620 (nx*ny + nx)	(1 - fx)(fy)(fz)
urc	(1,1,1)	621 (nx*ny + nx + 1)	(fx)(fy)(fz)

The initialize program segment initializes an offset array to speed up the interpolation process, because all field data arrays must have the same format and size. This array holds the relative array offset for each point in the 4 or 8 point interpolation box. In the above example the first four offsets would be 0, 1, 20, 21,...

The accel_adjust user program segment's subroutine initializes a weighting factor array. This array holds the value for each point's total weighting factor stored in the same order as the offset array above. In the above example, the first point's weighting factor would be $(0.7)(0.4)(0.1) = 0.028$.

Special issues with pressure and temperature fields

Corrections for local pressure and temperature are made by applying ratios of local values to the STP values. In order to avoid division by zero issues, the subroutine in the accel_adjust user program segment will always flag a fatal error if the local temperature or local pressure is calculated to be zero. Thus you must preload non-zero values for points in the temperature and/or pressure field arrays. If you get lazy, the user program will flag a fatal error and tell you that either local pressure or local temperature in the defined field was zero.

If you elect to only define one field (e.g. temperature), the user program will automatically use the user optioned static value for the other (e.g. pressure). It is your responsibility to insure that what you include and/or exclude in the way of fields makes physical sense. Garbage in – garbage out!

Special issues with bulk gas velocity fields

The initialize user program segment tests the VX_DEFS.DAT file for $n_x=0$. If $n_x = 0$ then the user program assumes that no bulk gas velocity field is defined and the user defined uniform velocity components are assumed (the contents of the other two velocity files are ignored).

Unlike pressure and temperature, bulk gas velocity is a vector field. If the potential array is 2D then only the VX_DEFS.DAT and VY_DEFS.DAT file must be defined, and VZ_DEFS.DAT file will be ignored. However, if the potential array is 3D then all three velocity component files must be supplied as non-zero. It is your responsibility to use the the potential array's dimensions and symmetry for the fields arrays. You can cheat, but you will either get garbage out or an array bounds error.

Remember: $\nabla \cdot \mathbf{v} = 0$

The most important point here is that it is your responsibility to insure that flow continuity is preserved in the velocity fields you create (e.g. the continuity equation above should hold if compression effects are minimal). Moreover, make sure that flow discontinuities do not occur across any mirrored boundaries. There should be no black holes or spontaneous mass sources within your flow field. Ignore this at your peril.

The other challenge facing the user program is the local velocity vector must be converted from absolute potential array coordinates back into workbench coordinates. This requires that the velocity vector be initially created in absolute potential array

coordinates. This vector is then transformed potential array volume coordinates (to include effects of mirroring and symmetry). The final transform involves reorienting the vector into SIMION's workbench coordinates (e.g. by using the **>WBO** user program command):

```
RCL vz_gu
RCL vy_gu
RCL vx_gu
>WBO
```

The specific transformation processes are discussed below by array type:

3D planar arrays

The transformation from absolute PA to PA volume coordinates in 3D planar symmetry potential arrays is reasonably simple. The conversion only requires the inclusion of mirroring effects. If the PA volume coordinate (e.g. `ion_px_gu`) is negative then the respective velocity component is multiplied by -1:

$$\text{If } (ion_px_gu < 0) v_x = -1.0v_x$$

$$\text{If } (ion_py_gu < 0) v_y = -1.0v_y$$

$$\text{If } (ion_pz_gu < 0) v_z = -1.0v_z$$

It is important that you make sure that flow discontinuities do not occur across mirrored boundaries. This creates the same sorts of problems as black holes and point mass sources (a garbage simulation).

2D planar arrays

The transformation from absolute PA to PA volume coordinates in 2D planar symmetry potential arrays is also simple. The conversion requires the inclusion of x and y mirroring effects (v_z WILL always be zero). If the PA volume coordinate (e.g. `ion_px_gu`) is negative then the respective velocity component is multiplied by -1:

$$\text{If } (ion_px_gu < 0) v_x = -1.0v_x$$

$$\text{If } (ion_py_gu < 0) v_y = -1.0v_y$$

$$v_z \equiv 0$$

Make sure that flow discontinuities do not occur across mirrored boundaries.

2D cylindrical symmetry arrays

The transformation from absolute PA to PA volume coordinates in 2D cylindrical symmetry potential arrays is more complex. First, the transformation requires the

inclusion of any x mirroring effects. If the PA volume coordinate `ion_px_gu` is negative then the `vx` component is multiplied by -1:

$$\text{If } (ion_px_gu < 0) v_x = -1.0 v_x$$

Second, the v_y velocity component in absolute PA coordinates must be converted into v_y and v_z velocity components in the PA volume coordinate system.

$$r = 1.0 \times 10^{-10} + \sqrt{ion_py_gu^2 + ion_pz_gu^2}$$

$$v_{yvol} = v_{yabs} \left(\frac{ion_py_gu}{r} \right)$$

$$v_{zvol} = v_{yabs} \left(\frac{ion_pz_gu}{r} \right)$$

The 1.0e-10 is added to `r` to prevent division by zero.

Field scaling factors

The SDS user program allows the user to scale the temperature, pressure and/or bulk gas velocity fields via the following three adjustable variable scale factors (default values = 1.0):

```
_Use_P_field_scale_factor
_Use_T_field_scale_factor
_Use_V_field_scale_factor
```

This provides an easy way to adjust the intensity of one or more of the spatially varying `T`, `P`, and `Vbulk gas` data fields without having to recreate the respective data file. Note: Be sure that your adjustments make physical sense or garbage will result. Moreover, these scale factors are only applied to the fields loaded in from the data files. This means that scale factors are not applied to pressure, temperature, and/or bulk gas velocity that are initialized to a volume wide constant value (no non-zero .DAT field file).

The `tstep_adjust` User Program Segment

When ions are flown in groups, the slowest ion drags back the pack. Normally this is not too much of a problem when flying at zero quality. However, there are times when field discontinuities or sharply changing field gradients aggravate this issue. The user program has an adjustable variable `_Use_min_time_step_usec` that can be used to interactively to speed things up. The normal value for this variable is 0.0 (no constraints). However, if things slow down too much, the user can access and change this variable from the Variables tab while the ions are flying to speed them up.

Issues Related to Using Magnetic Fields

SDS is capable of including magnetic fields in its simulations providing you observe a few rules and recognize the limitations involved. The first and most important rule is that SDS must always be attached to the magnetic potential array in a simulation if one is present. Thus if your simulation involves two overlapping potential arrays, one electrostatic (e.g. elect.pa0) and one magnetic (e.g. mag.pa), SDS must only be active on the magnetic array. This can be done by ensuring that the magnetic array is instance #1 since the accel_adjust and other_actions segments are by default configured to only run when the PA instance number, ion_instance variable, is 1.

The reason for this rule is that SIMION accumulates accelerations and magnetic accelerations are computed last. Thus if SDS were attached to the electrostatic array its viscous simulating accel_adjust user program segment would not be applied to the magnetic accelerations, because they have not been computed yet. **This is extremely important.** Check out the magnetic demo if you are unsure!

Magnetic fields with collisions

Think of magnetic fields as a magnetic effect decorrelated by collisions. If an ion is moving at a constant velocity in a magnetic field with no electrostatic forces acting on it, a uniform magnetic field will act to cause the ion to orbit in a circle. Further, if the ion is at thermal equilibrium with the other molecules in the gas it will have a velocity equal to the average thermal velocity described above. Thus its magnetic orbit will have a radius equal to the thermal cyclotron radius for the ion at that temperature and B field.

If the mean free path between collisions is very short relative to the ion's thermal cyclotron radius the arcs of the ion's trajectory circles become virtually straight lines. At this point there is almost no difference between the non-magnetic and magnetic trajectories. For example, a Nitrogen molecular ion in nitrogen gas at 25° C and atmospheric pressure has a mean free path of approximately 6.76×10^{-8} m. If it is in a 7 Tesla field (70,000 gauss) its thermal cyclotron radius is 1.979×10^{-5} m. The ratio $mfp/rcyc = 3.415867 \times 10^{-3}$ (a very small arc – basically a straight line). Thus the ion diffuses almost identically to a non-magnetic situation. SDS can probably predict the viscous **and** diffusion effects reasonably accurately.

However if the pressure was 0.001 atmospheres the ion would make slightly more than ½ thermal cyclotron orbit between collisions. SDS might estimate the viscous trajectory reasonably well, but it would tend to overestimate the diffusion. One must keep these issues in mind when conducting simulations with SDS. SDS provides the term. It is suggested that you run the magnetic demo as well as the more fundamental magnetic background demos (in the attached subdirectory) to gain a better appreciation for the issues involved.

You will note that SDS provides the value of B1 (gauss) in the output header for each flown ion at the beginning of each fly'm. B1 is the strength of the magnetic field required (given the user specified pressure and temperature (not fields)) to have the

SDS User Program Documentation

radius of the ion's thermal cyclotron orbit equal its mean free path. Thus when the magnetic field strength is B_1 , the value of $MFP/rcyc = 1.0$. This is sort of the threshold value for the magnetic field diffusion effects. Below this value magnetic diffusion effects are relatively minimal. Magnetic field strengths above B_1 can cause SDS to overestimate molecular diffusion, because ion motions start to become collections of significant circular arcs and/or circles.